

# Relatório de Arquitetura e Engenharia de Dados

## Processamento de Big Data e Visualização de Resíduos Sólidos Urbanos em Escala Local

**Status do Projeto:** Concluído com Sucesso • **Ambiente:** Kubernetes Local (Kind & Podman) • **Tecnologias:** Apache Spark, Streamlit, Python

### 1. Introdução e Escopo do Projeto

Este relatório técnico detalha a concepção, implementação e resolução de problemas de um pipeline de dados de ponta a ponta focado no **Diagnóstico de Resíduos Sólidos Urbanos do Brasil**. O desafio consistiu em extrair inteligência de uma base de dados bruta volumosa (*D\_{bruto}* *pprox 43 ext{ MB}* contendo mais de *119.000* registros), transformá-la de forma performática utilizando processamento distribuído, persistir os resultados de maneira segura e disponibilizá-los em um painel analítico interativo de baixa latência para o usuário final.

### 2. Arquitetura de Solução e Justificativa Tecnológica

A escolha das tecnologias baseou-se nos modernos padrões corporativos de Engenharia de Dados e Infraestrutura como Serviço (IaaS). Abaixo estão descritas as justificativas para cada componente da pilha tecnológica:

- **Podman (Substituto do Docker):** Utilizado como motor de containerização por sua arquitetura *daemonless* (sem um processo central rodando como root), o que eleva substancialmente o nível de segurança do ambiente local e otimiza o consumo de memória RAM do sistema operacional hospedeiro.
- **Kubernetes via Kind (Kubernetes in Docker/Podman):** Escolhido para simular com fidelidade matemática e arquitetural um ambiente de nuvem de produção (como AWS EKS ou Google GKE). O Kind permitiu orquestrar múltiplos microsserviços isolados com políticas estritas de rede e armazenamento.
- **Apache Spark (via Spark-on-K8s Operator):** Para a camada de extração e transformação (ETL). Em vez de processar os dados sequencialmente com scripts tradicionais, o Spark distribui a carga de trabalho em tarefas paralelas executadas por *Executors* dinâmicos, reduzindo o arquivo final consolidado para *1.1 ext{ MB}*.
- **Streamlit:** Adotado para a camada de consumo analítico devido à sua velocidade de desenvolvimento em Python puro e renderização reativa sob demanda, eliminando a necessidade de infraestruturas complexas de front-end.

### 3. O Fluxo de Implementação Passo a Passo

O projeto foi executado de forma estritamente linear, dividindo-se nas seguintes fases:

1. **Provisionamento do Cluster:** Inicialização do cluster Kubernetes através do Kind mapeado sobre o runtime do Podman.
2. **Arquitetura de Armazenamento Persistente:** Criação de um `PersistentVolume (PV)` e um `PersistentVolumeClaim (PVC)` para estabelecer um canal de comunicação via disco rígido compartilhado entre os Pods do Spark e o Pod do Site.
3. **Configuração de Segurança (RBAC):** Criação de uma `ServiceAccount` específica e vinculação de permissões através de um `ClusterRoleBinding`, permitindo que o Spark herde direitos para criar e destruir pods de processamento dinamicamente.
4. **Deploy do Processamento (Spark Job):** Submissão do manifesto do Spark Operator que disparou o Driver e coordenou o mapeamento e aglutinação dos dados do CSV bruto.
5. **Deploy da Aplicação Web:** Configuração de um Pod Python isolado para ler o resultado volumétrico tratado e expor a porta de rede.

### 4. Mapeamento de Erros e Resolução de Problemas (Troubleshooting)

A robustez final do ecossistema foi conquistada através do diagnóstico sistemático de falhas clássicas de engenharia de infraestrutura. A tabela abaixo resume as principais ocorrências mitigadas:

| Componente / Sintoma            | Causa Raiz Identificada                                                                        | Solução Aplicada                                                                         |
|---------------------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Spark Driver (CrashLoopBackOff) | Falta de autorização na API do Kubernetes para criar pods de execução.                         | Criação e amarração de <code>ServiceAccount</code> com escopo de controle.               |
| Script .sh corrompido           | Quebra de linha gerada no padrão Windows (CRLF) em sistema operacional Linux.                  | Conversão explícita do encoding de terminação para <b>LF</b> no editor.                  |
| Site falhando após ~80s         | O Streamlit buscava o arquivo de inicialização no caminho incorreto <code>/app/app.py</code> . | Correção do script de largada para ler do volume montado em <code>/dados/app.py</code> . |
| Warning de PIP desatualizado    | Imagem base com versão de pacote anterior ao ciclo atual.                                      | Injeção de <code>pip install --upgrade pip</code> automatizado na esteira de boot.       |

### ⚠ Estudo de Caso Crítico: O Erro do Caminho do Arquivo (Causa Mortis do Pod)

O sintoma observado via `kubect1 get pods -w` apontava que o container do site era criado com sucesso, mantinha-se em execução por 77 segundos (tempo exato do download de bibliotecas) e repentinamente alternava para o estado de `Error`. A inspeção minuciosa dos logs expôs a falha:

```
=== Inicializando o Streamlit ===  
Usage: streamlit run [OPTIONS] [TARGET] [ARGS]...  
Error: Invalid value: File does not exist: /app/app.py
```

Como o ecossistema do Kubernetes utiliza volumes isolados, o arquivo copiado via Pod Ajudante encontrava-se em `/dados/app.py`. A reconfiguração da linha de execução para refletir o caminho absoluto correto estabilizou a aplicação definitivamente.

## 5. Fontes Consultadas e Documentações de Referência

Para assegurar as melhores práticas vigentes na comunidade internacional de tecnologia, a engenharia do projeto baseou-se nos seguintes guias oficiais de referência:

- **Documentação Oficial do Apache Spark (Spark on Kubernetes):** Padrões de submissão de jobs, gerenciamento de perfis de recursos (núcleos de CPU e limites de memória alocados aos executores).
- **Guia de Arquitetura do Streamlit (Deployment Server Configuration):** Parametrização de portas de escuta ( `--server.port` ) e binds de interface de rede ( `--server.address=0.0.0.0` ) para exposição correta em ambientes de container de microsserviços.
- **Kubernetes Core Documentation (Concepts - Storage):** Configuração do ciclo de vida de volumes persistentes e tratamento de concorrência de leitura/escrita de arquivos por múltiplos agentes de rede.

## 6. Conclusão

O projeto atingiu o estado de maturidade máxima planejada. O pipeline demonstra estabilidade operacional contínua ( `ext{Restarts} = 0` ), tempo de resposta imediato para o usuário final no endereço tunelado local e estabelece uma arquitetura elástica reutilizável, validando os conceitos avançados de Big Data aplicados.